Contract Risk Analysis via Context-Conditioned LLM Adaptation

Aarav Singh Luthra, Devyansh Choudhary, Saksham Bali, Uday Sodhi

Abstract

Legal contracts, particularly B2B agreements such as licensing agreements, often contain clauses that impose financial or legal obligations that may not be immediately clear to non-expert users. In this project, we develop a contract analysis system that extends a pretrained instruction-tuned Large Language Model (LLM) with structured, clause-level risk extraction capabilities. The system first preprocesses uploaded contracts through deterministic document parsing and clause segmentation to produce semantically coherent clause units. The system then performs structured risk **classification** conditioned on user role and contract domain using parameter-efficient fine-tuning via Low-Rank Adaptation (LoRA) with 4-bit quantisation (QLoRA). The model produces **categorical risk levels (Low, High)** along with concise natural-language explanations tailored to the specified user context. We further integrate a Retrieval-Augmented Generation (RAG) module to support grounded cross-questioning over uploaded contracts. The system is evaluated using classification accuracy, precision, recall and macro F1 score, across both a zero-shot baseline and the fine-tuned model, alongside human evaluation of RAG-generated responses for contextual grounding.

1. Introduction

Legal contracts are complex textual artifacts that encode rights, obligations, liabilities, and procedural constraints. Businesses frequently engage with licensing agreements and other B2B contracts that define IP usage rights, payment obligations, termination conditions, and liability exposure between commercial parties. Interpreting clause-level implications in such documents requires careful attention to language that may signal risk depending on the reader's role and priorities.

Instruction-tuned Large Language Models (LLMs) provide strong general-purpose reasoning capabilities and can generate natural-language explanations for textual inputs. However, out-of-the-box models are not explicitly trained to produce structured, clause-level risk assessments conditioned on user role and contextual attributes.

In this project, we aim to adapt a pretrained instruction-tuned LLM to perform structured, context-conditioned legal risk extraction. The system:

1. Preprocesses uploaded contracts and segments them into coherent clauses.
2. Performs clause-level risk labelling conditioned on user role, contract domain, and risk tolerance.
3. Produces structured outputs containing the predicted risk level and a concise explanation.
4. Additionally supports grounded cross-questioning via retrieval augmentation.

Our objective is to study how parameter-efficient adaptation and explicit context conditioning can enable a pretrained language model to internalize user-aware legal risk interpretation in a controlled and measurable manner.

2. Problem Formulation

Let a contract be represented as an ordered set of clauses:

$$C = \{c_1, c_2, \ldots, c_n\}$$

where each clause c_i is a semantically coherent unit extracted from a document belonging to domain D.

Each clause is analyzed relative to a user context consisting of user role U (e.g., licensor, licensee)

The core modeling objective is to learn:

$$f(c_i, U) \rightarrow (r_i, e_i)$$

where

- r_i is a categorical risk level for clause c_i
- e_i is a natural-language explanation describing the potential implication of the clause for the specified user context

The risk level belongs to one of the two categories:

- **Low**
- **High**

Note: The project initially explored a three-class setup (Low / Medium / High), but Medium-risk labels frequently introduced ambiguity during annotation and reduced consistency across role-conditioned judgments. To improve supervision quality and model reliability, the final system adopts binary risk classification using only Low and High labels.

For interactive querying, we define:

$$g(C, U, Q) \rightarrow A$$

where Q is a user question and A is a grounded answer generated using retrieved clauses from C.

The system consists of two logical modules sharing a fine-tuned LLM core:

(1) Risk Flagging Module—structured clause-level risk extraction

(2) Cross-Questioning Module—retrieval-conditioned generation.

3. Related Work

3.1 Legal NLP

Legal NLP research has explored domain-adaptive pretraining, clause classification, and document-level inference. LegalBERT (Chalkidis et al., 2020) demonstrates benefits of legal-domain pretraining.

The CUAD dataset (Hendrycks et al., 2021) provides clause-level annotations for commercial contracts. Recent work has extended CUAD in several directions. CUAD-SL (2024) restructured the dataset as a single-label classification benchmark across 23 clause categories, where fine-tuned DeBERTa achieved 87.8% accuracy compared to 67.2% for zero-shot GPT-4, confirming that domain fine-tuning outperforms prompting alone on legal classification tasks. ContractEval (Liu et al., 2025) benchmarked 19 LLMs — including LLaMA 3.1 and Mistral — on clause-level legal risk identification using CUAD, finding that open-source models at the 7–8B scale are viable candidates for legal tasks when combined with targeted fine-tuning. A parallel line of work has studied LLM-assisted annotation quality: Savelka (2023) demonstrated that zero-shot GPT models applied without explicit rubrics produce inconsistent labels grounded in pretraining patterns rather than any defined legal standard, directly motivating the rubric-constrained annotation design used in this project. Additionally, the ALeaseBERT benchmark (Leivaditi et al., 2020) introduced a dataset of 179 lease agreements manually annotated for 14 entity types and 17 red flag categories by legal professionals, providing a precedent for domain-specific, expert-guided annotation of risk-relevant contract elements.

ContractNLI (Koreeda and Manning, 2021) frames contract reasoning as a natural language inference problem. These works provide datasets and modelling strategies relevant to legal text processing but do not explicitly study user-conditioned clause-level risk classification.

3.2 Instruction-Tuned LLMs and Parameter-Efficient Adaptation

Instruction-tuned models such as LLaMA (Touvron et al., 2023) and Mistral (Jiang et al., 2023) demonstrate strong zero-shot reasoning capabilities. Low-Rank Adaptation (LoRA) (Hu et al., 2022) enables efficient fine-tuning by inserting low-rank adapter matrices into transformer layers while freezing base parameters. This approach allows domain-specific adaptation without full retraining.

3.3 Retrieval-Augmented Generation

Retrieval-Augmented Generation (Lewis et al., 2020) combines dense retrieval with language model generation to improve grounding and factual consistency. This framework can be applied to contract analysis to support clause-anchored question answering.

Recent legal-domain RAG research provides concrete implementation guidance. LegalBench-RAG (Pipitone & Alami, 2024) established that Recursive

Character Text Splitting (RCTS) outperforms fixed-size chunking for legal text retrieval, and that general-purpose reranking models can reduce retrieval performance when applied to legal corpora without modification. A subsequent NLLP 2025 study introduced Summary-Augmented Chunks (SAC), where each clause chunk is enriched with a short LLM-generated semantic summary before indexing, significantly improving retrieval precision on CUAD contracts. The same study reported hallucination rates of 58–80% for general-purpose LLMs on legal tasks, reinforcing the importance of grounded retrieval for the cross-questioning module in this project. ACORD (Wang et al., 2025), a clause retrieval and drafting dataset built directly on CUAD contracts, found that a bi-encoder retriever paired with an LLM reranker achieves the best retrieval performance, which informs the architecture of our RAG module.

4 Methodology

4.1 Dataset Construction

4.1.1 Clause Extraction and Annotation Pipeline.

To build a role-conditioned legal risk classification dataset, we constructed a clause-level annotation pipeline over License Agreements sourced from the CUAD dataset. Since CUAD provides clause-type annotations but does not provide user-role-conditioned risk labels (Low/High) for Licensor and Licensee perspectives, a custom dataset construction process was required. The final pipeline consists of two core stages: deterministic clause extraction and LLM-assisted risk labelling, followed by manual verification for quality control. The final fine-tuning dataset contains 4,075 annotated examples and is used for binary risk classification (Low / High) using Llama 3.1 8B Instruct.

Step 1: Clause Extraction from Contract PDFs
Raw License Agreement PDFs are first processed using a deterministic clause extraction pipeline implemented in clauseextraction.py. The objective is to convert long unstructured legal documents into clean clause-level units suitable for independent annotation and model training keeping original structural boundaries present in the contract using rule-based segmentation.

The extraction pipeline operates in five stages:
1. PDF Parsing
Contract PDFs are parsed using pdfplumber, which extracts raw text line-by-line along with positional metadata such as page number, line height, vertical spacing, font size, and boldness. This layout information is essential for identifying clause boundaries and section headers accurately.

2. Clause Boundary Detection
The extracted lines are grouped into clause blocks using rule-based legal formatting patterns. A new block is created when the system detects:

- page breaks
- large vertical gaps between lines
- bold uppercase section headers
- top-level numbered clauses (e.g., 1., 2.)
- decimal subsections (e.g., 2.1, 3.2)
- legal subparts such as (a), (b), (i), (ii), a., i.

Boilerplate content such as page numbers, headers, footers, signature blocks, and exhibit markers is removed during this stage.

3. Sub-Clause Splitting
Continuation lines belonging to the same clause are merged, while inline sub-clauses such as:

- (a)... (b)... (c)...
- (i)... (ii)...
- 2.1, 2.2, 2.3

are split into separate records for fine-grained annotation.
Each sub-clause maintains references to its parent clause using:

- parent_clause_id
- parent_clause_header

This preserves legal hierarchy while allowing clause-level independent analysis.

4. Clause Enrichment
Each extracted clause is stored with additional metadata including:

- clause ID
- section header
- page number
- document position (early / middle / late)
- party references (Licensor, Licensee, both parties)
- token count
- subpart label
- parent clause references (if applicable)

Very short clauses with low informational value are removed to reduce annotation noise.

5. Output Generation
The final clauses are saved as:

- JSONL records (one clause per line)

- .txt files with blank-line-separated clauses for annotation input

This format directly supports the next LLM-assisted labelling stage.
This deterministic extraction process ensures stable, reproducible, and legally consistent clause segmentation across contracts.

Step 2: LLM-Assisted Risk Labelling
After clause extraction, each clause is annotated using the pipeline implemented in annotate.py, which uses the Groq API with GPT-OSS-120B to generate structured legal risk labels. The goal of this stage is to create supervised training data for clause-level risk classification by assigning risk labels separately for both Licensor and Licensee perspectives.

Initially, the annotation task was designed as a **three-class classification problem** with the labels:

- Low Risk
- Medium Risk
- High Risk

The model was instructed to evaluate each clause based on what legal or financial cost, restriction, or obligation it created for each party.

Initial Three-Class Risk Annotation
A detailed legal rubric was embedded directly into the system prompt so that the LLM would apply predefined rules instead of making subjective judgments.
The prompt defined:

- **High Risk** for severe legal exposure such as:
  - uncapped liability or indemnification
  - unilateral termination without cause
  - revocable licenses without protection
  - IP ownership transfer to the non-creating party
  - severe operational restrictions
  - unilateral amendment rights
  - automatic renewal without reasonable opt-out
- **Medium Risk** for clauses that created obligations or limitations requiring attention, but without severe open-ended exposure, such as:
  - capped liability
  - termination with notice
  - post-termination obligations
  - audit rights
  - confidentiality obligations surviving termination
- **Low Risk** for standard procedural or protective clauses such as:
  - definitions
  - notices
  - severability clauses
  - signature blocks
  - headings
  - counterparts clauses
  - interpretation rules

The annotation rule required the model to always assign the **highest applicable trigger**, preventing averaging across multiple signals.
Each clause was labelled independently for:

- Licensor
- Licensee

since the same clause may represent different levels of risk depending on which party is affected.

Class Imbalance and Shift to Binary Classification
After annotation and initial fine-tuning experiments, a major issue observed was the model collapsing toward predicting the Medium-risk class.
In early three-class experiments (Low / Medium / High), Llama 3.1 8B showed near-perfect recall for Medium-risk clauses while almost entirely failing to correctly identify Low-risk clauses, even though Low-risk clauses formed the majority of the dataset. This indicated that the model was treating Medium as a default prediction rather than learning meaningful distinctions between the three categories.
Additionally, a clear performance gap was observed between Licensor and Licensee perspectives on the same set of clauses, showing that party viewpoint significantly affects risk interpretation and that zero-shot predictions were unable to handle this role-based asymmetry reliably.
The Medium category also introduced annotation ambiguity, since many clauses could reasonably be interpreted as either Low or Medium depending on safeguards such as notice periods, liability caps, and contractual limitations.
To improve supervision quality and model reliability, the final system was converted into a binary classification task:

- Low Risk
- High Risk

Medium-risk cases were merged into High for training purposes, allowing the model to focus on distinguishing safe clauses from clauses that require legal attention.
Instead of re-running the entire annotation pipeline through the API, all previously annotated **Medium-risk** labels were directly merged into the **High-risk** class during dataset preparation. No re-annotation using annotate.py was performed. Existing Medium labels were reassigned to High, and only necessary

reasoning adjustments were reviewed manually where required to maintain consistency with the final binary risk schema.
This significantly improved label consistency and made role-conditioned fine-tuning more stable and effective.

Final Annotation Process
The final prompt follows a strict two-step reasoning structure.
1. Scratchpad Reasoning
Before assigning labels, the model must explicitly reason through:

- what the clause costs or restricts for the Licensor
- what the clause costs or restricts for the Licensee
- whether any High-risk trigger applies
- whether safeguards such as caps, notice periods, or limitations reduce severity

This forces the model to reason using the rubric instead of relying on surface-level pattern matching.
2. Structured JSON Output
After reasoning, the model returns structured JSON output:

```
{
  "clause_text": "...",
  "parties": {
    "Licensor": {
      "risk": "High | Low",
      "explanation": "..."
    },
    "Licensee": {
      "risk": "High | Low",
      "explanation": "..."
    }
  }
}
```

This creates role-conditioned labelled examples where each clause receives separate labels and explanations for both parties.

Pipeline Reliability
The annotation script also includes:

- resume support for interrupted runs
- automatic retry for rate-limit errors
- skipping clauses exceeding context length
- JSON validation checks
- status logging (success, json_error, api_error)

This ensures reliable large-scale annotation across thousands of clauses and allows continuous dataset generation without manual interruption

Step 3: Manual Verification and Final Dataset Preparation
A subset of generated labels is manually reviewed to ensure consistency with the predefined rubric, especially for clauses where role-based differences are subtle or legally ambiguous.
Incorrect labels are corrected manually, and repeated error patterns are used to refine prompt instructions and improve future annotation quality.
The final dataset contains **4,075 clause-role examples**, with each example containing:

- clause text
- user role (Licensor / Licensee)
- binary risk label (Low / High)
- short legal explanation

This dataset is used for QLoRA fine-tuning of **Llama 3.1 8B Instruct**, which serves as the final deployed model for clause-level contract risk classification.

Initially, when the CUAD annotation pipeline was in process, we conducted initial baseline experiments using the Legal Indian Contract Clauses Dataset that contains:

- Clause text
- Clause type
- Risk level

This dataset does not contain user-role annotations and is therefore used only to obtain preliminary baseline results for clause-level risk classification.

4.1.2 Risk Schema

The system performs role-conditioned binary legal risk classification at the clause level. Each clause is evaluated independently from the perspective of the relevant party—primarily the Licensor and Licensee in License Agreements—and assigned one of two labels:

- Low Risk
- High Risk

As discussed in Section 4.1.1, the project initially explored a three-class setup (Low / Medium / High). However, due to class imbalance, ambiguity in Medium-risk labels, and the model's tendency to over-predict the Medium class during early experiments, the final system was converted into a binary classification task.

Medium-risk cases were merged into High, allowing the model to focus on distinguishing safe clauses from clauses requiring legal attention. This improved annotation consistency and fine-tuning stability.

The final risk labels are generated using an explicit legal rubric embedded directly into the annotation prompt, ensuring that classification follows deterministic legal rules rather than subjective LLM judgment.

Low Risk

A clause is labelled Low Risk when it creates minimal legal or financial exposure for the specified party, or when it primarily grants protection, rights, or procedural safeguards rather than obligations.

If a clause protects a party, it is considered Low for that party. Similarly, if a clause grants a party a right or option, the risk applies to the counterparty rather than the beneficiary.

High Risk

A clause is labelled High Risk when it creates significant legal, financial, or operational exposure for the specified party, especially when obligations are severe, unilateral, open-ended, or difficult to mitigate.

The system follows one strict rule:

If any High-risk trigger is present, then High and otherwise, Low.

The highest applicable trigger always determines the final label, ensuring consistency across documents

4.2 Inference Pipeline

4.2.1 Preprocessing and Clause Extraction

Before modeling, uploaded documents undergo a preprocessing pipeline:

1. Document parsing (PDF → raw text with position and font metadata.).
2. Boilerplate filtering — page numbers, signature lines, exhibit markers, Roman numeral page indicators, and formatting artifacts are removed using a set of predefined regex patterns.
3. Deterministic clause boundary detection using vertical gap thresholds (gaps exceeding 1.5× the median line height trigger a new block), font metadata (bold short uppercase lines are treated as section headers), and pattern matching for numbered headings (e.g. '1.', '2.1'), Section/Article prefix indicators, and lettered or Roman numeral subparts ((a), (i), i., a.).
4. Continuation merging — blocks not ending with sentence-final punctuation are merged forward with the next block to recover wrapped text. Inline sub-clause splitting — blocks containing embedded sub-clause sequences (e.g. '…end. (a) text… (b) text…') are split into individual clause units using a sentence-boundary-aware regex.
5. Clause enrichment — each extracted clause is annotated with its nearest section header, relative document position (early/middle/late), party references found in the text (licensor, licensee, either party), subpart label, parent clause ID for sub-clauses, and token count.

This pipeline produces stable and reproducible clause-level inputs for modeling.

4.2.2 Base Model and LoRA Fine-Tuning

We use a pretrained instruction-tuned Llama-3.1-8B-Instruct as the base model. Fine-tuning is performed using QLoRA with 4-bit quantization for memory efficiency.

The base model weights are fully frozen; only, low-rank adapters inserted into the four attention projection layers (q_proj, k_proj, v_proj, o_proj) are trainable. With LoRA rank r = 8 and a = 16, this yields 0.0848% i.e. 6,815,744 trainable parameters relative to the full 8B parameter model.
Training is conducted for 3 epochs using a learning rate of $2x10^{-4}$

Training examples follow a structured instruction format:

User Role: [role]
Clause: [text]
Output:
Risk Level: [label]
Explanation: [text]

Cross-entropy loss is computed over output tokens only. This enables the model to learn structured, context-aware risk extraction without architectural modification.

4.2.3 Risk Flagging Module

For inference, the pipeline operates as follows:

1. The user uploads a contract
2. The contract is segmented into clauses
3. Each clause is passed to the fine-tuned LLM with user context
4. Structured outputs are parsed and stored
5. Clauses are returned with predicted risk.

4.2.4 Retrieval-Augmented Generation (RAG) Module

The RAG module supports grounded follow-up questioning after a contract has been analyzed. It operates independently of the classification pipeline and uses a pre-built knowledge base of annotated CUAD clauses as its retrieval corpus.

Knowledge Base and Indexing. The RAG database consists of 500 annotated clauses from the CUAD dataset, stored as structured records containing the raw clause text, per-party risk labels, and per-party explanations for both Licensor and Licensee. Each clause is converted into an enriched text string before embedding, concatenating its risk labels with the clause text in the format: "Licensor Risk: X. Licensee Risk: Y. Clause: [text]". This enriched representation is embedded rather than raw clause text, providing the embedding model with richer semantic signal that includes risk context alongside clause content.

Embeddings are computed using BAAI/bge-base-en-v1.5, a sentence transformer model designed for dense retrieval over technical and legal text. All embeddings are L2-normalised before indexing. The index uses FAISS IndexFlatIP — flat inner product search over unit-normalised vectors, which is mathematically equivalent to cosine similarity. The index is built once and persisted to disk so subsequent runs load it directly without re-encoding. Index construction processes all 500 clauses in batches of 32 on GPU.

Retrieval. For each user question, the query string is prefixed with "Represent this legal clause for retrieval: " before embedding — a BGE-specific requirement that significantly improves retrieval quality. The prefixed query is embedded with the same model and searched against the FAISS index using exact nearest-neighbour search, returning the top-3 most similar clauses by cosine similarity score. Role-specific framing is applied at retrieval time: the same retrieved clause is presented with either its Licensor or Licensee risk label and explanation depending on the user's selected role, so a clause that is high risk for the Licensee but low risk for the Licensor is presented appropriately to each.

4.2.5 System Architecture

The full system is implemented as a FastAPI backend deployed within a Google Colab runtime, exposed to the browser via an ngrok tunnel, and accessed through a standalone HTML frontend.

Backend. The backend (backend.py) exposes two primary endpoints. POST /analyze accepts a multipart form upload containing the PDF and user role. It runs clause extraction, classifies each clause through the fine-tuned model with role context, stores results in a server-side session, and returns the full clause list with risk labels and explanations. POST /ask accepts a JSON body with the user question and role, retrieves the top-3 reference clauses from the FAISS index, builds a structured prompt, generates a response, updates conversation history, and returns the answer with source snippets. A GET /health endpoint verifies model and RAG readiness. CORS middleware is enabled to allow the local HTML file to call the API across the ngrok tunnel.

Classification Prompt. For each clause during /analyze, the model receives a two-message prompt in LLaMA 3.1's chat template format. The system message establishes the legal risk analyst persona, instructs the model to focus on practical consequences for the specified party, and enforces a strict two-line output format (Risk Level / Explanation). The user message contains the party role, clause text prefixed by its section header, and the classification question. Generation uses greedy decoding (do_sample=False, temperature=None) for deterministic outputs across identical clauses.

Chat Prompt. For follow-up questions, the model receives a multi-turn prompt containing: a system message with role instructions and a filtered contract analysis summary (high-risk clauses only, plus any clause numbers explicitly mentioned, to stay within token budget); the full conversation history in alternating user/assistant format; and the current user turn containing the top-3 RAG-retrieved reference clauses with role-specific risk labels and explanations followed by the user's question. MAX_SEQ_LEN is set to 8192 to prevent truncation on large contracts. Chat generation uses nucleus sampling (temperature=0.7, top_p=0.9).

Session State. The server maintains an in-memory session storing the initial classification results, selected role, and conversation history. Session state resets on each new /analyze call.

Frontend and Deployment. The frontend is a single self-contained HTML file with no framework dependencies. It presents a role selection screen before document upload, renders clause results as expandable colour-coded risk cards, and provides a chat interface displaying responses alongside retrieved source snippets. The backend runs as a daemon thread inside Colab. An ngrok tunnel maps a public HTTPS URL to localhost:8000, which is patched into the HTML's API base URL and the file is downloaded locally for use in any browser

4.3 Experiments

To evaluate the effectiveness of the proposed system, we conducted the following experiments:

Experiment 1: Zero-Shot Baseline

We evaluate LLaMA 3.1 8B Instruct and Mistral 7B Instruct in a zero-shot setting without any fine-tuning or role context on our binary Low/High risk schema. This experiment establishes baseline performance and identifies which model demonstrates stronger out-of-the-box legal reasoning ability.

Both models are evaluated on a stratified sample of 200 clauses. With two risk classes, a random baseline would yield 50% accuracy; both models achieve approximately 51–52% accuracy, indicating that general-purpose LLMs possess limited capability for legal risk classification and require domain-specific adaptation for reliable performance. LLaMA 3.1 8B is selected for fine-tuning based on its slightly higher Macro F1 score (0.446 vs 0.428), which better reflects per-class performance than accuracy alone given class imbalance.

Based on these results, LLaMA 3.1 8B was selected as the base model for all subsequent fine-tuning experiments. While the accuracy difference between the two models is marginal, LLaMA 3.1 8B's stronger Macro F1 score indicates better per-class discrimination — a more meaningful signal than overall accuracy when class distributions are uneven. Given that running both models through the full fine-tuning pipeline was not practical under compute constraints, this zero-shot evaluation served as the decisive selection criterion.

Model	Accuracy	Macro F1
Zero-shot LLaMA 3.1 8B	51.5%	0.446
Zero-shot Mistral 7B Instruct	52.0%	0.428

Experiment 2: Projection Layers and learning rate

We conducted experiments to identify an efficient fine-tuning configuration under compute constraints. All exploratory runs were performed on carefully selected subsets of ~200–300 samples to enable rapid iteration while maintaining representative clause coverage. All experiments were designed to operate within realistic compute constraints, guiding trade-offs in model depth, data balancing, and training configuration.

We first applied LoRA to 7 transformer layers (attention + MLP). Although deeper adaptation could improve reasoning capacity, it was computationally expensive with limited empirical gains in early runs. We therefore restricted adaptation to 4 layers, targeting only attention projections (q_proj, k_proj, v_proj, o_proj), achieving a better performance–compute trade-off.

To address class imbalance, we evaluated undersampling and oversampling strategies. Undersampling did not improve High-risk recall, as the absolute number of High-risk examples remained limited while useful Low-risk training signal was removed. Aggressive oversampling introduced redundancy without consistent gains. The final setup used moderate oversampling of High-risk clauses: 903 → 1,590 (factor ≈ 1.76), reducing the class ratio from 2.64:1 to 1.5:1.

A learning rate of 2×10^{-4} yielded stable convergence over 3 epochs.

The final configuration, 4-layer attention-only LoRA, learning rate 2×10^{-4}, and moderate oversampling, was adopted for all subsequent experiments.

5 Evaluation

5.1 Evaluation Metrics

Structured Risk Prediction

Categorical risk level outputs are parsed and evaluated using Accuracy, Precision, Recall, and Macro F1-score. Evaluation is conducted separately for the zero-shot baseline and the role-conditioned QLoRA fine-tuned model, and separately from both Licensor and Licensee perspectives.

RAG Evaluation

RAG performance is evaluated through human evaluation — responses generated with retrieval are assessed for whether the answer is grounded in the

retrieved reference clauses and directly addresses the question asked.

5.2 Baseline Results

For primary evaluation, we assess zero-shot LLaMA 3.1 8B Instruct on our test set of 826 clauses from the role-conditioned CUAD-derived dataset, using the binary Low/High risk schema. The model achieves 64.4% accuracy with a macro F1 of 0.572. Performance is notably asymmetric across classes — the model achieves reasonable performance on Low risk clauses (F1: 0.748, precision: 0.766, recall: 0.730) but struggles significantly on High risk clauses (F1: 0.395, precision: 0.374, recall: 0.419), reflecting a bias toward predicting Low risk and an inability to reliably identify clauses with genuine legal exposure. The test set contains 597 Low and 229 High examples, and the performance gap between classes is consistent with this imbalance.

```
Accuracy: 0.6440677966101694

Classification Report:

              precision    recall  f1-score   support

         LOW     0.7663    0.7303    0.7479       597
        HIGH     0.3735    0.4192    0.3951       229

    accuracy                         0.6441       826
   macro avg     0.5699    0.5748    0.5715       826
weighted avg     0.6574    0.6441    0.6500       826
```

As an earlier diagnostic, we also run zero-shot LLaMA 3.1 8B on an annotated subset of the CUAD dataset consisting of 117 clauses from License Agreements, evaluated separately from both the Licensor and Licensee perspectives. From the Licensor perspective, the model achieves 40.2% accuracy with a macro F1 of 0.373, while from the Licensee perspective it achieves 23.1% accuracy with a macro F1 of 0.243. In both cases, the model collapses towards predicting Medium risk, achieving near-perfect recall for that class while almost entirely failing to identify Low risk clauses despite them being the majority class. Additionally, the notable performance gap between the two perspectives on the same set of clauses suggests that party viewpoint influences classification in ways the zero-shot model does not handle well, motivating the role-conditioned fine-tuning approach described in the next stage of experiments.

Licensee Perspective (CUAD Subset)

```
— Per-class metrics ————————————————————————————————

              precision    recall  f1-score   support

         Low      1.000     0.098     0.179        51
      Medium      0.316     0.939     0.473        33
        High      0.786     0.333     0.468        33

    accuracy                         0.402       117
   macro avg      0.701     0.457     0.373       117
weighted avg      0.747     0.402     0.343       117
```

Licensor Perspective (CUAD Subset)

```
— Per-class metrics ————————————————————————————————

              precision    recall  f1-score   support

         Low      1.000     0.067     0.126        89
      Medium      0.180     0.947     0.303        19
        High      0.273     0.333     0.300         9

    accuracy                         0.231       117
   macro avg      0.484     0.449     0.243       117
weighted avg      0.811     0.231     0.168       117
```

5.3 Fine-Tuned Model Results

The role-conditioned QLoRA fine-tuned LLaMA 3.1 8B is evaluated on the same 826-clause test set used for the zero-shot baseline, enabling direct comparison. Results are reported overall and separately for Licensor and Licensee perspectives (413 examples each).

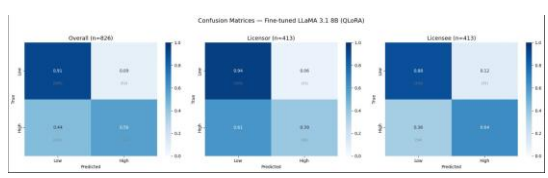
Overall Performance

```
==================================================
OVERALL PERFORMANCE (val set)
==================================================
Accuracy: 0.815

              precision    recall  f1-score   support

         Low       0.84      0.91      0.88       597
        High       0.71      0.56      0.63       229

    accuracy                           0.81       826
   macro avg       0.78      0.74      0.75       826
weighted avg       0.81      0.81      0.81       826
```



The fine-tuned model achieves 81.5% overall accuracy with a macro F1 of 0.750, compared to 64.4% accuracy and macro F1 of 0.572 for the zero-shot baseline — an improvement of 17.1 percentage points in accuracy and 17.8 points in macro F1. The most significant gain is on High risk classification, where F1 improves from 0.395 to 0.630, reflecting the model's improved ability to identify clauses with genuine legal exposure after domain adaptation. Low risk F1 also improves from 0.748 to 0.880. The confusion matrix confirms the model now correctly classifies 91% of Low risk clauses and 56% of High risk clauses overall, compared to the zero-shot model which was heavily biased toward Low predictions.

Per-Role Analysis

```
=====================================================
LICENSOR (n=413)
=====================================================
Accuracy: 0.835
              precision    recall  f1-score   support

         Low       0.87      0.94      0.90       336
        High       0.59      0.39      0.47        77

    accuracy                           0.84       413
   macro avg       0.73      0.66      0.69       413
weighted avg       0.82      0.84      0.82       413
```

```
LICENSEE (n=413)
=====================================================
Accuracy: 0.794
              precision    recall  f1-score   support

         Low       0.81      0.88      0.84       261
        High       0.76      0.64      0.70       152

    accuracy                           0.79       413
   macro avg       0.78      0.76      0.77       413
weighted avg       0.79      0.79      0.79       413
```

Performance differs meaningfully between the two roles, which is expected given the asymmetric nature of contract risk. From the Licensee perspective the model achieves 79.4% accuracy (macro F1: 0.770), with Low F1 of 0.840 and High F1 of 0.700. From the Licensor perspective it achieves 83.5% accuracy (macro F1: 0.690), with Low F1 of 0.900 but a notably lower High F1 of 0.470.

The Licensor High risk performance is weaker primarily due to class imbalance within the Licensor test split — only 77 of 413 Licensor examples are High risk (18.7%), compared to 152 of 413 for Licensee (36.8%). With fewer High risk training signals from the Licensor perspective, the model achieves only 39% recall on Licensor High risk clauses, meaning it misses over half of genuinely high-risk clauses from that perspective. This is visible in the confusion matrix, where 61% of true Licensor High risk clauses are misclassified as Low. The Licensee perspective is better balanced and accordingly yields stronger High risk recall at 64%.

5.4 RAG Evaluation

The cross-questioning module is evaluated through qualitative human assessment rather than automated metrics, as the goal of the RAG module is conversational grounding rather than classification accuracy. For evaluation, a set of follow-up questions were posed after uploading sample contracts, covering queries such as explaining why a specific clause was flagged as high risk, identifying the most concerning clauses for a given role, and asking what changes would reduce risk in a particular clause.

Responses were assessed on two criteria: whether the answer directly addressed the question asked, and whether the response was visibly grounded in the retrieved reference clauses shown as sources. In all evaluated cases, the retrieved sources were topically relevant to the question — for example, questions about termination clauses consistently retrieved reference clauses involving termination conditions, and questions about indemnification retrieved clauses with liability language. Generated responses incorporated the risk labels and explanations from retrieved clauses rather than producing generic answers.

The primary limitation observed is that the fine-tuned model, having been trained specifically for structured clause classification, produces less fluent conversational responses compared to a general-purpose chat model. Responses are accurate and grounded but tend toward structured enumeration rather than natural prose. This is an expected consequence of using a classification-fine-tuned model for generation and is noted as a direction for future improvement.

5.5 Analysis and Discussion

Dataset Construction and Annotation Strategy

The primary concern with dataset construction was ensuring that risk labels reflect a consistent and reproducible definition of risk rather than the internal biases of whatever model generates them. General-purpose LLMs are not explicitly trained to perform structured clause-level legal risk classification, they assign risk based on patterns learned during pretraining which are inconsistent, opaque, and not grounded in any explicit standard. To address this, we first define a risk rubric ourselves that specifies exact criteria for what constitutes Low, Medium, and High risk for each user role — later collapsed to binary Low/High due to class imbalance — and then constrain the LLM strictly to applying these criteria. This reduces the LLM's role to rule application rather than rule invention, ensuring that labels are traceable back to our own definitions. Manual verification of a subset further catches cases where even rule guided annotation produces inconsistent outputs, particularly on ambiguous clauses where legal language is deliberately vague.

Importance of Role Conditioning

The zero shot results exposed the most fundamental limitation of a general-purpose model for this task, it assigns identical risk to identical clauses regardless of user role. A termination clause is not inherently risky, it is risky for the employee and routine for the employer, and no amount of domain fine tuning alone can capture this asymmetry because the risk is not in the clause text but in the relationship between the clause and the user. The role-conditioned fine-tuning experiment directly addresses this by explicitly injecting user role into every training example. The results confirm the intervention is effective — the fine-tuned model achieves 79.4% accuracy from the Licensee perspective and 83.5% from the Licensor perspective, compared to 64.4% overall for the zero-shot baseline. The per-role performance gap (macro F1: 0.770 Licensee vs 0.690 Licensor) further reflects genuine asymmetry in how risk is distributed across roles in licensing agreements, with Licensor High risk clauses being rarer and harder to learn from.

Class Imbalance and High Risk Detection

The most consistent limitation across both baseline and fine-tuned results is weaker performance on High risk clauses relative to Low. This is a direct consequence of class imbalance — 597 of 826 test examples (72.3%) are Low risk. The fine-tuned model improves High risk F1 from 0.395 to 0.630 overall, but the Licensor perspective remains weaker (High F1: 0.470) because only 77 of 413 Licensor examples are High risk. The model correctly identifies 56% of High risk clauses overall but misses 44%, which in a legal context means genuine risk exposure goes unflagged. Future work should explore oversampling High risk examples during fine-tuning or applying a lower classification threshold for High risk to improve recall at the cost of some precision.

Model	Accuracy	Macro F1	Low F1	High F1
Zero-shot baseline	0.644	0.572	0.748	0.395
Fine-tuned (overall)	0.815	0.750	0.880	0.630
Fine-tuned (Licensee)	0.794	0.770	0.840	0.700
Fine-tuned (Licensor)	0.835	0.690	0.900	0.470

Limitations

The primary limitation of the current system is weaker High-risk detection, particularly from the Licensor perspective, largely driven by class imbalance in the training data. This imbalance reflects an inherent property of the dataset—Licensors are typically less exposed to risk-inducing clauses in License Agreements, resulting in a limited number of High-risk examples for that role. Consequently, the model misses approximately 44% of true High-risk clauses overall, which is a meaningful gap for a legal risk tool.

Additionally, the system is currently scoped only to License Agreements from CUAD, and generalization to other B2B contract types has not been evaluated. The fine-tuned model was also trained for structured classification rather than conversational generation, which limits the fluency of RAG chat responses.

A further limitation is the strict computational constraints under which the system was developed. Fine-tuning was restricted to a small subset of layers using parameter-efficient methods; training a larger portion of the model or using deeper adaptation could

potentially yield substantial improvements in reasoning capability and High-risk detection performance.

Future Work

Several directions are identified for future improvement. First, applying a lower classification threshold for High risk predictions would improve recall on the more consequential class. Second, extending the annotation pipeline and fine-tuning to other B2B contract types beyond License Agreements would improve generalization. Third, replacing the fine-tuned model in the RAG chat module with a general-purpose instruction-tuned model would produce more fluent conversational responses while keeping the fine-tuned model for classification only. Fourth, the RAG retrieval could be strengthened by pre-retrieving reference clauses similar to each classified clause at upload time, rather than retrieving only at question time.

6. Conclusion

In this project, we propose a framework for context-conditioned contract risk analysis using instruction-tuned large language models.

The system performs clause-level risk classification conditioned on user role and contract domain, currently scoped to Licensor and Licensee roles within License Agreements, with scope for extension to broader B2B agreement types in future work, producing structured outputs consisting of categorical risk levels and natural-language explanations.

Our methodology combines deterministic contract preprocessing, parameter-efficient model adaptation using QLoRA, and a structured annotation pipeline guided by a predefined risk rubric. Evaluation results confirm that role-conditioned QLoRA fine-tuning yields substantial improvements over the zero-shot baseline — overall accuracy improves from 64.4% to 81.5% and macro F1 from 0.572 to 0.750 — demonstrating that explicit role conditioning enables the model to internalize the asymmetric nature of contract risk across different parties.

Overall, this work aims to demonstrate how structured context conditioning can enable LLMs to perform interpretable and user-aware legal risk analysis.

References

Ilias Chalkidis, Manos Fergadiotis, Prodromos Malakasiotis, Nikolaos Aletras, and Ion Androutsopoulos. 2020. LEGAL-BERT: The muppets straight out of law school. In Findings of EMNLP.

Dan Hendrycks, Collin Burns, Anya Chen, and Spencer Ball. 2021. CUAD: An expert-annotated NLP dataset for legal contract review. In NeurIPS Datasets and Benchmarks.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In ICLR.

Yuta Koreeda and Christopher Manning. 2021. ContractNLI: A dataset for document-level natural language inference for contracts. In Findings of EMNLP.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In NeurIPS.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, et al. 2023. LLaMA: Open and efficient foundation language models. arXiv:2302.13971.

O'Connell, E., Duffy, W., McCarroll, N., Sloan, K., Curran, K., McNamee, E., Clist, A., and Brammer, A. 2025. Cost–benefit analysis of deploying shallow, deep learning and generative models for legal text classification. Artificial Intelligence and Law, 1–35.

Savelka, J. 2023. Unlocking practical applications in legal domain: Evaluation of GPT for zero-shot semantic annotation of legal texts. Proceedings of the Nineteenth International Conference on Artificial Intelligence and Law, 447–451.

Leivaditi, S., Rossi, J., and Kanoulas, E. 2020. A benchmark for lease contract review. arXiv preprint arXiv:2010.10386.

Liu, S., Li, Z., Ma, R., Zhao, H., and Du, M. 2025. ContractEval: Benchmarking LLMs for clause-level legal risk identification in commercial contracts. arXiv preprint arXiv:2508.03080.

Pipitone, N. and Houir Alami, G. 2024. LegalBench-RAG: A benchmark for retrieval-augmented generation in the legal domain. arXiv preprint arXiv:2408.10343.

NLLP Workshop. 2025. Towards reliable retrieval in RAG systems for large legal datasets. ACL Anthology.

Wang, S. H., Zubkov, M., Fan, K., Harrell, S., Sun, Y., Chen, W., Plesner, A., and Wattenhofer, R. 2025. ACORD: An expert-annotated retrieval dataset for legal contract drafting. Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 24739–24762.

Individual Contributions

Aarav Singh Luthra

Aarav built and ran the zero-shot evaluation pipeline for both LLaMA 3.1 8B and Mistral 7B Instruct, producing the baseline performance metrics that guided model selection and determined which model the team would carry forward into fine-tuning.

In the earlier phase of the project, the dataset construction and annotation approach was designed to operate across multiple contract domains but did not generalize well due to limited coverage within each domain. To address this, Aarav redesigned and implemented a more robust end-to-end annotation pipeline tailored to a focused setting of B2B contracts, specifically License Agreements, where multiple documents within a single domain could be processed consistently. This included developing the annotation script, designing the structured prompt, and formalizing the risk rubric with trigger-based decision rules. He also performed final dataset sampling, class balancing through oversampling of High-risk clauses, and final model training using the selected QLoRA configuration, ensuring stable fine-tuning performance on the binary Low/High risk classification task.

Devyansh Choudhary

Devyansh designed the role-dependent risk rubric that defines how risk levels are assigned across different user roles and contract types, forming the conceptual foundation of the annotation pipeline.

As the project scope evolved, he conducted detailed analysis of CUAD and related legal NLP datasets to understand how domain-specific datasets are used effectively in prior work. This informed the shift from a multi-domain setup to a focused B2B contract setting. He also contributed to the manual verification of annotated data, ensuring consistency with the rubric and identifying cases where the model's outputs required correction.

In addition, he worked extensively on frontend development, helping design the user-facing interface for contract upload, clause-wise risk visualization, and interactive follow-up questioning through the RAG-based chat module, improving usability and overall system presentation.

Saksham Bali

Saksham developed the document preprocessing and clause segmentation pipeline that converts raw legal contracts into structured, semantically coherent clause units, ensuring clean and consistent inputs for both the annotation process and model training.

The initial version of the preprocessing pipeline was designed for diverse contract formats and did not generalize reliably across domains. Saksham refined and improved this pipeline to better handle multiple documents within a single domain improving consistency in clause extraction. He also implemented additional data processing components, including PDF-to-text extraction and dataset balancing to ensure equal representation of risk categories.Additionally, he contributed to reviewing related work and refining the overall data annotation pipeline for improved robustness. He also contributed to experimentation with model training and inference, helping evaluate different fine-tuning configurations and improving model performance through iterative testing.

Uday Sodhi

Uday worked on early-stage dataset construction and annotation workflows that explored role-conditioned labelling across multiple contract domains. While this approach provided initial insights, it faced limitations in scalability and consistency across heterogeneous document types.

Building on this, Uday focused on evaluation and experimentation, implementing the baseline evaluation pipeline using the processed dataset. He conducted zero-shot experiments and analysed model performance across Licensor and Licensee perspectives, contributing to the empirical validation of the system and helping identify the need for a more domain-focused annotation strategy. He also handled making of RAG and deployment of the complete

system by connecting the FastAPI backend with the standalone HTML frontend, managing Colab-based serving, ngrok integration, API endpoint testing, and preparing the required deployment files for end-to-end project execution.

Project GitHub Repository:

https://github.com/Saksham-Bali/Advanced_NLP_proj